

CareKeeper — เอกสารสรุปโครงการฉบับสมบูรณ์

เอกสารนี้รวบรวมข้อมูลทั้งหมดของโปรเจกต์ CareKeeper ไว้ในที่เดียว สำหรับใช้ประกอบการนำเสนอ/ส่ง อาจารย์/ส่งต่อทีมงานใหม่ ครอบคลุมโครงสร้างโปรเจกต์ สถาปัตยกรรม โลจิกที่ใช้โมดูลและฟังก์ชันการทำงาน ตัวแปรแวดล้อม (.env) การ deploy บน Raspberry Pi และข้อจำกัดที่ยังต้องพัฒนาต่อ (ปรับปรุงล่าสุด 24 สิงหาคม 2569: เปลี่ยนเซนเซอร์ SpO2 เป็น MAX30102, เพิ่มเซนเซอร์อุณหภูมิ DS18B20, auto-detect พอร์ตเครื่องวัดความดัน, หน้าข้อมูลเครื่อง (Device ID) และปุ่มปิดหน้าจอ)

สารบัญ

1. ภาพรวมโครงการ (Project Overview)
2. วัตถุประสงค์และขอบเขต
3. สถาปัตยกรรมระบบ (Architecture)
4. โครงสร้างไฟล์ (Project Structure)
5. Libraries / Dependencies
6. ตัวแปรแวดล้อม (Environment Variables — .env)
7. รายละเอียดโมดูลและฟังก์ชัน (Module & Function Reference)
8. การจัดการ Thread และ Concurrency
9. Data Flow: จากบัตรประชาชนถึง Backend
10. หน้าจอของระบบ (UI Screens) — Template รูป
11. การติดตั้งและรันโปรแกรม (Setup & Run)
12. การ Deploy บน Raspberry Pi 5 (Kiosk Mode)
13. การทดสอบ (Testing)
14. การจัดการข้อผิดพลาดและการแจ้งเตือน
15. ภาคผนวก (Appendix)
16. ข้อจำกัดปัจจุบันและงานที่ควรทำต่อ (Known Limitations / Roadmap)

1. ภาพรวมโครงการ (Project Overview)

CareKeeper เป็นระบบส่วนติดต่อผู้ใช้แบบกราฟิก (GUI) สำหรับเครื่องตรวจวัดสัญญาณชีพ (Vital Signs Kiosk) ที่ทำงานบน Raspberry Pi 5 พัฒนาด้วยภาษา Python 3 และ framework PySide6 (Qt for Python) ออกแบบมาสำหรับหน้าจอสัมผัสขนาด 1010 × 503 พิกเซล

ระบบช่วยให้ผู้ใช้งาน (เช่น เจ้าหน้าที่สาธารณสุขหรือประชาชนทั่วไป) สามารถ:

- อ่านข้อมูลผู้รับบริการจากบัตรประชาชนไทย (Thai National ID Card) ผ่านเครื่องอ่านบัตร (Smart Card Reader / PC-SC)
- กรอกรายละเอียดประชาชนเองในกรณีเครื่องไม่สามารถอ่านข้อมูลบัตรได้
- วัดค่าความดันโลหิต (SYS/DIA) และชีพจร (Pulse) จากเครื่องวัดความดันผ่าน serial, ออกซิเจนในเลือด (SpO2) จากเซนเซอร์ MAX30102 ผ่าน I2C และอุณหภูมิร่างกายจากเซนเซอร์ DS18B20 ผ่าน 1-Wire — เชื่อมต่ออุปกรณ์จริงครบทุกค่าแล้ว
- แสดงผลค่าที่วัดได้แบบเรียลไทม์ พร้อมแยกสีตามชนิดข้อมูลเพื่อให้อ่านง่าย
- สรุปผลการวัดทั้งหมดในหน้าเดียว และส่งข้อมูล (JSON) ไปยัง Backend API ของโรงพยาบาล/ระบบ Telemedicine
- แสดงข้อมูลย้อนหลัง 4 รายการล่าสุดของผู้รับบริการ
- ทำงานต่อเนื่องได้แม้ไม่มีอินเทอร์เน็ต (Offline-first) ด้วยระบบคิวสำรองข้อมูล
- ทนทานต่ออุปกรณ์ที่เชื่อมต่อไม่สำเร็จ ด้วยระบบ retry + auto-disable
- ดูข้อมูลเครื่อง (IP / Hostname / MAC และคำสั่ง ssh) ได้จากหน้าจอ เพื่อเข้าเครื่องระยะไกลหลังเปลี่ยนเครือข่าย
- ปิดหน้าจอแอปกลับสู่เดสก์ท็อปได้เมื่อต้องตั้งค่าเครื่องหน้างาน โดย systemd จะไม่เปิดแอปซ้ำ

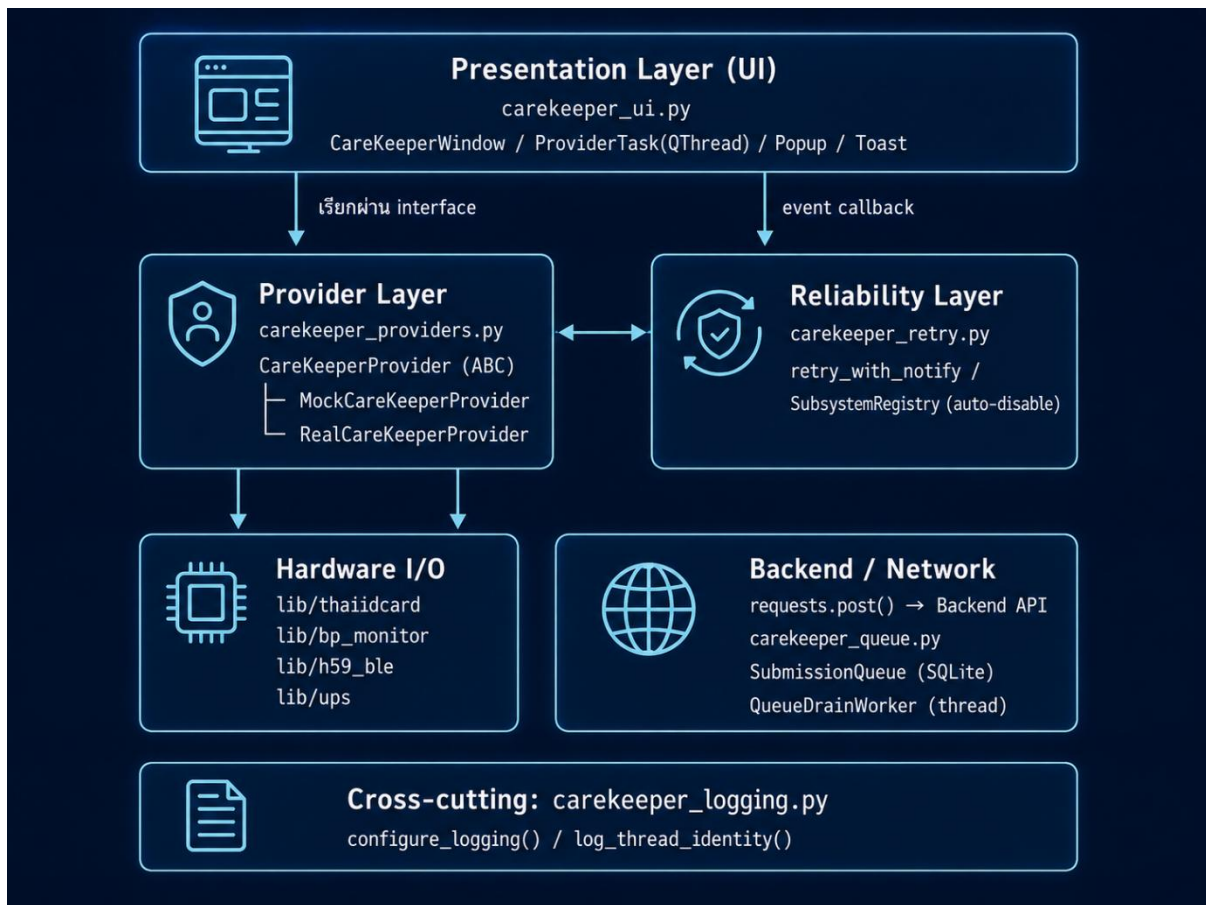
โปรเจกต์ถูกออกแบบให้แยกชิ้นการทำงาน (UI / Provider / Retry / Queue / Logging) อย่างชัดเจน ทำให้สามารถพัฒนาและทดสอบ UI ด้วยข้อมูลจำลอง (Mock Mode) ได้โดยไม่ต้องพึ่งอุปกรณ์จริงตลอดเวลา และสลับไปใช้อุปกรณ์จริง (Real Hardware Mode) ได้ทันทีเมื่อพร้อม deploy บน Raspberry Pi

2. วัตถุประสงค์และขอบเขต

หัวข้อ	รายละเอียด
เป้าหมายหลัก	รวมการทำงานของอุปกรณ์ตรวจสอบสุขภาพหลายชนิดไว้ในโปรแกรมเดียว ลดความซับซ้อนสำหรับผู้ใช้ทั่วไป
กลุ่มผู้ใช้งาน	เจ้าหน้าที่/อาสาสมัครสาธารณสุข, ประชาชนที่มาใช้ตรวจสุขภาพ
แพลตฟอร์มเป้าหมาย	Raspberry Pi 5 + จอสัมผัส 1010x503 พิกเซล, Raspberry Pi OS (labwc/Wayland)
ขอบเขตที่รองรับแล้ว	อ่านบัตรประชาชน, วัดความดัน/ชีพจร, วัด SpO2 (MAX30102), วัดอุณหภูมิ (DS18B20), auto-detect พอร์ตเครื่องวัดความดัน, ส่งข้อมูล Backend, offline queue, retry/auto-disable, Wi-Fi/Bluetooth management, แสดง IP/Device ID, ปิดหน้าจอกลับเดสก์ท็อป, kiosk boot
ขอบเขตที่ยังไม่สมบูรณ์	GET API ข้อมูลย้อนหลังจริงจาก Backend, log แบบไฟล์/journald, การทดสอบกับฮาร์ดแวร์จริงแบบอัตโนมัติ (hardware-in-the-loop)

3. สถาปัตยกรรมระบบ (Architecture)

ระบบแบ่งเป็นเลเยอร์หลัก 5 ชั้น ทำงานร่วมกันดังนี้:



หลักการออกแบบสำคัญ

- **Dependency Inversion ระหว่าง UI กับ Hardware**: `CareKeeperWindow` ไม่เรียก hardware โดยตรง แต่เรียกผ่าน interface `CareKeeperProvider` ทำให้สลับ Mock ↔ Real ได้โดยไม่แก้โค้ด UI
- **Non-blocking UI**: งานที่รอ I/O (อ่านบัตร, วัดค่า, ส่ง API, สแกน Wi-Fi/Bluetooth) ทำงานในแยกจาก main thread เสมอ
- **Resilience by design**: ทุกการเรียกอุปกรณ์ห่อด้วย `retry_with_notify` (มีรุ่น async สำหรับงาน BLE) และมี `SubmissionQueue` กันข้อมูลหายเมื่อออฟไลน์
ส่วนงานที่ต้องรอค่าจากร่างกาย เช่น SpO2 และอุณหภูมิ จะหา "ค่านี้" เองก่อนคืนค่า
- **Config over code**: ค่าที่เปลี่ยนบ่อย (API URL, API key, serial port, BLE address) อยู่ใน `.env` ไม่ hardcode ในโค้ด

4. โครงสร้างไฟล์ (Project Structure)

Care_Keeper/

———	main_demo.py	# entry point: รัน GUI ด้วย MockCareKeeperProvider
———	main_real.py	# entry point: รัน GUI กับอุปกรณ์จริงบน Raspberry Pi
———	carekeeper_ui.py	# หน้าจอหลักทั้งหมด (Scan / Dashboard / Summary), widget, popup, dialog
———	carekeeper_providers.py	# provider interface + Mock/Real implementation + Backend API client
———	carekeeper_retry.py	# retry_with_notify (+ รัน async) + SubsystemRegistry (auto-disable)
———	carekeeper_queue.py	# SubmissionQueue (SQLite offline queue) + QueueDrainWorker
———	carekeeper_logging.py	# ตั้งค่า logging กลาง + log thread identity
———	carekeeper_style.py	# stylesheet (สี, ฟอนต์, องค์ประกอบหน้าจอ) แบบ QSS
———	requirement.txt	# runtime Python dependencies
———	requirements-dev.txt	# เพิ่ม pytest / pytest-qt / pytest-mock / pytest-cov สำหรับ dev
———	.env	# ค่าจริง (gitignore, ห้าม commit)
———	.gitignore	
———	sensor_tests/	# สคริปต์ทดสอบอุปกรณ์แยกเดี่ยวก่อนเปิด GUI จริง (รันมือ ไม่ใช่ pytest)
———	_project_path.py	# helper: เติม project root เข้า sys.path ให้สคริปต์ import lib/ ได้
———	idcard.py	# ทดสอบอ่านบัตรประชาชน
———	BP.py	# ทดสอบเครื่องวัดความดัน
———	H59_BLE.py	# ทดสอบอุปกรณ์ SpO2 แบบ BLE (ของเดิม เกือบไว้อ้างอิง)
———	battery.py	# ทดสอบ UPS/แบตเตอรี่
———	ble_scanner.py	# ทดสอบค้นหา (scan) Bluetooth device
———	lib/	# โมดูลเชื่อมต่ออุปกรณ์จริง (hardware drivers)
———	bp_monitor.py	# driver เครื่องวัดความดันผ่าน serial (pyserial) + รหัสสาเหตุที่วัดไม่ได้
———	temp_sensor.py	# [ใหม่] driver เซนเซอร์อุณหภูมิ DS18B20 ผ่าน 1-Wire (หาค่านี้เอง)
———	ups.py	# driver UPS HAT ผ่าน I2C (battery/power status)
———	thaiidcard/	
———	card.py	# ThaiIDCard class: เชื่อมต่อ/อ่านข้อมูลบัตร (pyscard)
———	apdu.py	# ค่าคงที่ APDU command สำหรับสื่อสารกับ smart card
———	init.py	
———	spo2_max30102/	# [ใหม่] driver เซนเซอร์ SpO2 MAX30102 ผ่าน I2C (แทน H59 BLE)
———	max30102.py	# คุยกับ register/FIFO ของชิป MAX30102 ผ่าน smbus/smbus2
———	hrcalc.py	# อัลกอริทึมคำนวณ HR/SpO2 ของผู้ผลิตเซนเซอร์ (ใช้ตามต้นฉบับ)
———	spo2_monitor.py	# SpO2Monitor: ตรวจสอบว่ามี, หาค่านี้ แล้วคืนค่ามัธยฐาน
———	__init__.py	
———	h59_ble/	# [เดิม] SpO2/Heart-rate ผ่าน Bluetooth LE (bleak) — เกือบไว้อ้างอิง
———	device.py	# H59Device: เชื่อมต่อ BLE, ส่ง/รับคำสั่ง, keepalive
———	spo2.py	# SpO2Reader: parse ค่า SpO2 จาก notification

- heart_rate.py # HeartRateReader: parse ค่าชีพจรจาก notification
 - README.md
- style/ # asset ฟอนต์และไอคอน
 - NotoSansThai-Regular.ttf
 - IBMPlexSansThai-Regular.ttf
 - Asimov-MwEn.otf
 - id-card-svgrepo-com.svg
- conf/ # ตัวอย่าง systemd unit สำหรับ deploy แบบ kiosk บน Pi
 - carekeeper.service # auto-run main_real.py ตอนบูต + RestartPreventExitStatus=42
 - carekeeper-restart.service / .timer # reboot อัตโนมัติทุกที 3
 - carekeeper-journal.conf # จำกัดขนาด systemd journal log
- docs/ # เอกสารประกอบโครงการ
 - HealthLink.docx / .pdf # เอกสารฉบับนี้
 - CareKeeper_Full_Documentation.md
 - carekeeper-pi5-setup-runbook.md # ขั้นตอนตั้งเครื่อง Pi 5 ใหม่ทีละ Phase
 - test-report.md # รายงานผลการทดสอบซอฟต์แวร์
- data/ # SQLite ของ offline queue (สร้างเองตอนรัน, ไม่ commit)
 - carekeeper_queue.db
- tests/ # ชุดทดสอบอัตโนมัติ (pytest, 231 test cases)
 - conftest.py # fixture กลาง (reset SubsystemRegistry, neutralize retry backoff sleep)
 - fakes/ # fake provider/serial/ble/max30102/requests สำหรับ mock ในเทสต์
 - test_providers_bp.py / _spo2.py / _idcard.py / _history.py / _wifi_bt.py / _send_data.py
 - test_spo2_stability.py # [ใหม่] การหาค่านิ่งของ SpO2 (MAX30102) ก่อนคืนค่าสุดท้าย
 - test_queue.py / test_queue_worker.py
 - test_retry_helper.py
 - test_threading.py
 - test_ui_status_disabled.py / test_ui_submit_flow.py
 - test_ui_bp_cooldown.py # [ใหม่] การลือกปุ่มวัดความดันระหว่างนับถอยหลัง cooldown
 - test_ui_elided_label.py # [ใหม่] ย่อข้อความยาวในแถบข้อมูลผู้รับบริการ
 - test_device_status_fields.py
 - test_h59_packets.py / _notify_parsers.py / _reader_flows.py / _device.py
 - test_thaiidcard.py # unit test สูตรแปลงวันเกิด พ.ศ., decode TIS-620, โปรโตคอล APDU
 - test_ups.py # unit test ถอดรหัส register I2C ของ UPS HAT
 - test_bp_monitor_parsing.py # unit test parser โปรโตคอล serial ของเครื่องวัดความดัน

5.ไลบรารีและแพ็คเกจที่เกี่ยวข้อง (Libraries / Dependencies)

5.1 Runtime dependencies (requirement.txt)

Library	หน้าที่ในโปรเจกต์
PySide6	Qt for Python — ใช้สร้าง GUI ทั้งหมด (QMainWindow, QThread, widget, stylesheet)
pyqtgraph	เตรียมไว้สำหรับแสดงกราฟ/พล็อตข้อมูลสัญญาณชีพแบบเรียลไทม์ (เผื่อขยายในอนาคต)
bleak	Bluetooth Low Energy client แบบ cross-platform — ใช้สแกน/เชื่อมต่ออุปกรณ์ BLE ในเมนู Bluetooth ของเครื่อง และไดรเวอร์ H59 เดิม
pyserial	สื่อสารกับเครื่องวัดความดันโลหิตผ่านพอร์ต serial (USB-to-serial) และใช้ serial.tools.list_ports ในการ auto-detect พอร์ต
pyscard	เชื่อมต่อเครื่องอ่านบัตรสมาร์ทการ์ด (PC/SC) เพื่ออ่านข้อมูลบัตรประชาชนไทย
requests	เรียก HTTP POST/GET ไปยัง Backend API (ส่งผลตรวจ, ดึงข้อมูลย้อนหลัง)
python-dotenv	โหลดค่าตัวแปรแวดล้อมจากไฟล์ .env เข้าสู่ os.environ
numpy	[ใหม่] คำนวณสัญญาณจากเซนเซอร์ SpO2 MAX30102 (hrcalc.py และการหาค่านี้ใน spo2_monitor.py)
smbus2	[ใหม่] คุย I2C กับชิป MAX30102 บน Raspberry Pi (ใช้แทน smbus ของระบบเมื่อไม่มี)

5.2 Dev dependencies (requirements-dev.txt)

Library	หน้าที่ในโปรเจกต์
pytest	ตัว test runner หลักของโปรเจกต์
pytest-qt	เทสต์ widget/event ของ PySide6 (จำลอง signal/slot, event loop)
pytest-mock	สร้าง mock/patch แบบสะดวกใน unit test
pytest-cov	วัดความครอบคลุมของโค้ด (code coverage) สำหรับรายงานผลการทดสอบ

5.3 System packages (Raspberry Pi OS)

Library	หน้าที่ในโปรเจกต์
pcscd, pcsc-tools, libccid	daemon และเครื่องมือสำหรับอ่านเครื่องอ่านบัตรสมาร์ทการ์ดผ่าน PC/SC
libpcsclite-dev, swig	จำเป็นสำหรับ compile ตอน pip install pycard
python3-smbus, i2c-tools	สื่อสารผ่าน I2C bus — ใช้ทั้ง UPS HAT (แบตเตอรี่) และเซนเซอร์ SpO2 MAX30102 (ต้องเปิด I2C ใน raspi-config; ตรวจสอบด้วย i2cdetect -y 1)
bluetooth, bluez	Bluetooth stack ของ Linux สำหรับ BLE และคำสั่ง bluetoothctl
dtoverlay=w1-gpio (ใน /boot/firmware/config.txt)	[ใหม่] เปิด 1-Wire สำหรับเซนเซอร์อุณหภูมิ DS18B20 — lib/temp_sensor.py อ่านค่าจาก /sys/bus/w1/devices/28-*/w1_slave

6. ตัวแปรแวดล้อม (Environment Variables — .env)

ค่าที่เปลี่ยนบ่อยตามอุปกรณ์/สภาพแวดล้อมจริงถูกย้ายออกจากโค้ดไปไว้ในไฟล์ .env

ตัวแปร	ใช้ทำอะไร	ตัวอย่างค่า
CAREKEEPER_API_URL	endpoint สำหรับ ส่งผลตรวจ (JSON) ไปยัง Backend ด้วย HTTP POST	https://.../api/v2/device/add_health
CAREKEEPER_API_KEY_HEADER	ชื่อ HTTP header ที่ใช้ส่ง API key ไปกับ request	api-key
CAREKEEPER_API_KEY	ค่า API key จริง สำหรับยืนยันตัวตนกับ Backend	test (ต้องเปลี่ยนก่อนใช้งานจริง)
CAREKEEPER_HISTORY_API_URL	endpoint สำหรับ ดึงข้อมูลย้อนหลังของผู้รับบริการ	https://.../api/v2/device/health_history
CAREKEEPER_HISTORY_PATIENT_ID_PARAM	ชื่อ query parameter ที่ใช้ส่งเลขบัตรประชาชนไปกับ request ประวัติ	patient_id
CAREKEEPER_HISTORY_MAC_PARAM	ชื่อ query parameter ที่ใช้ส่ง MAC address ของเครื่องไปกับ request ประวัติ	mac
CAREKEEPER_BP_PORT	serial port ที่เครื่องวัดความดันต่ออยู่ — ใส่ auto หรือเว้นว่างไว้ได้ ระบบจะค้นหาพอร์ต USB-serial ให้เอง และถ้าใส่พอร์ตไว้แต่ถอดสายไปแล้ว ก็จะมี auto-detect แทน	/dev/ttyUSB0 หรือ auto
CAREKEEPER_SPO2_FINGER_IR_THRESHOLD	[ใหม่] ระดับสัญญาณ IR ขั้นต่ำที่ถือว่า มีนิ้ววางบนเซนเซอร์ MAX30102 ปรับต่อเครื่องได้ด้วยคำสั่ง python -m lib.spo2_max30102.spo2_monitor --raw	10000 (ค่าปริยาย)
CAREKEEPER_H59_DEVICE_NAME	[เดิม] ชื่ออุปกรณ์ BLE ของเครื่องวัด SpO2 H59 — ไม่ได้ใช้แล้วหลังเปลี่ยนไปใช้ MAX30102	H59_D105
CAREKEEPER_H59_DEVICE_ADDRESS	[เดิม] BLE address (UUID) ของอุปกรณ์ H59 — ไม่ได้ใช้แล้วหลังเปลี่ยนไปใช้ MAX30102	EC9C2DA6-F503-4660-0ABB-3ABFA92F9E5D

7. รายละเอียดโมดูลและฟังก์ชัน (Module & Function Reference)

7.1 carekeeper_ui.py — Presentation Layer

ไฟล์นี้ใหญ่ที่สุดในโปรเจกต์ รับผิดชอบ UI ทั้งหมด แบ่งเป็นกลุ่มดังนี้

Helper functions (module-level)

ฟังก์ชัน	หน้าที่
<code>_style_asset(name)</code>	คืน path เต็มของไฟล์ asset ใน style/
<code>_load_font_family(...), _load_app_font(...), _load_number_font()</code>	โหลดฟอนต์ไทย/ตัวเลขแบบกำหนดเองเข้า Qt Application
<code>_tinted_icon(name, size, color)</code>	โหลดไอคอนแล้วแต่งสีตามธีม

Core classes

Class	สืบทอดจาก	หน้าที่
VitalState	—	เก็บสถานะค่าที่วัดล่าสุด (SYS/DIA/PUL/SpO2/Temp) ระหว่างใช้งาน
ProviderTask	QThread	รัน callable ใดๆ (เช่น <code>provider.measure_blood_pressure</code>) ใน background thread แล้วยิง signal กลับ main thread เมื่อเสร็จ/ผิดพลาด
RetryNotifier / QueueDrainNotifier	QObject	เป็นสะพาน signal จาก <code>carekeeper_retry</code> / <code>carekeeper_queue</code> (ซึ่งรันอยู่ใน thread อื่น) กลับเข้า Qt event loop อย่างปลอดภัย
WifiIndicator, BluetoothIndicator, BatteryIndicator	QWidget	widget วาดไอคอนสถานะ Wi-Fi/Bluetooth/แบตเตอรี่ พร้อมรับคลิกเพื่อเปิดเมนูจัดการ
PowerButton	QWidget	ปุ่มเปิดเมนูระบบ: reboot / shutdown / ดูข้อมูลเครื่อง (Device ID) / ปิดหน้าจอ
ToastLabel, PopupOverlay	QLabel / QWidget	กล่องข้อความแจ้งเตือนกลางหน้าจอ (toast) พร้อม auto-dismiss
ElidedLabel	QLabel	[ใหม่] ย่อข้อความที่ยาวเกินความกว้างช่องด้วยจุดไข่ปลา (...) แทนที่จะตัดแฉกสั้น ใช้กับชื่อผู้รับบริการที่ยาวจนชนเลขบัตรบนจอแคบของ Pi (ยังอ่านค่าเต็มได้จาก <code>text()</code>)
CareKeeperWindow	QMainWindow	หน้าต่างหลักของทั้งแอป ประกอบทุกหน้าจอ (Scan/Dashboard/Summary) และ orchestrate การเรียก provider ทั้งหมด

เมธอดสำคัญของ CareKeeperWindow

Method	หน้าที่
<code>_start_task(action, ...)</code>	จุดกลางในการสร้าง ProviderTask แล้ว task.start() เพื่อรันงาน hardware/backend แบบไม่ block UI
<code>_start_network_task(...)</code>	เหมือน <code>_start_task</code> แต่เฉพาะงาน Wi-Fi/Bluetooth และกันการกดสแกน/เชื่อมต่อซ้ำกัน
<code>_request_device_status()</code>	สั่งอ่านสถานะ Battery/Wi-Fi/Bluetooth เป็นรอบผ่าน worker
<code>_build_scan_page()</code>	สร้างหน้าจอที่ 1: อ่านบัตรประชาชน
<code>_build_dashboard_page()</code>	สร้างหน้าจอที่ 2: วัดค่าสัญญาณชีพ
<code>_build_summary_page()</code>	สร้างหน้าจอที่ 3: สรุปผลการวัด
<code>_read_card()</code>	เริ่ม flow อ่านบัตรประชาชนผ่าน provider
<code>_show_manual_cid_entry() / _submit_manual_cid()</code>	เปิด/ยืนยัน popup กรอกเลขบัตรด้วยตนเอง (ตรวจว่าเป็นตัวเลข 13 หลัก)
<code>_format_manual_cid_input(text)</code>	จัดรูปแบบเลขบัตรระหว่างพิมพ์ และปฏิเสธอักขระที่ไม่ใช่ตัวเลข
<code>_measure_bp() / _measure_spo2() / _measure_temperature()</code>	เริ่มวัดแต่ละค่าผ่าน provider โดยเช็คว่า subsystem นั้นไม่ได้ถูก auto-disable ไว้
<code>_start_bp_cooldown(seconds, succeeded) / _bp_cooldown_tick()</code>	[ใหม่] ล็อกปั๊มวัดความดันพร้อมนับถอยหลัง 60 วินาที หลังวัดสำเร็จ และหลังเครื่องแจ้ง BP_ERROR (ซึ่งเครื่องจะล็อกตัวเองอยู่แล้ว) เมื่อครบเวลาปั๊มจะกลับไปเป็นสถานะเดิมก่อนเริ่มนับ
<code>_submit_data()</code>	รวมค่าที่วัดได้เป็น JSON payload (รวม mac จาก read_device_mac()) แล้วส่งไป Backend ผ่าน provider
<code>_on_queue_drain_success/ failure(...)</code>	callback เมื่อ background worker ส่งข้อมูลค้างใน offline queue สำเร็จ/ล้มเหลว
<code>_on_retry_attempt/ giveup(...)</code>	callback แสดงสถานะเมื่อระบบ retry กำลังพยายามใหม่ หรือ auto-disable subsystem
<code>_open_wifi_selector() / _open_bluetooth_selector()</code>	เปิดหน้าต่างเลือกและเชื่อมต่อ Wi-Fi/Bluetooth
<code>_show_device_id()</code>	[ใหม่] แสดงหน้าต่างข้อมูลเครื่อง: IP (จาก provider.get_ip_address()), Hostname, MAC และคำสั่ง ssh ที่พร้อมใช้ เพื่อให้หาเครื่องเจอหลังเปลี่ยนเครือข่าย

<code>_exit_ui()</code>	[ใหม่] ถ้ามียืนยันแล้วปิดหน้าจอแอปกลับเดสก์ท็อป โดยออกด้วย exit code 42 (EXIT_CODE_CLOSE_DISPLAY) ซึ่ง systemd ตั้ง RestartPreventExitStatus ไว้ จึงไม่เปิดแอปซ้ำ
<code>_do_reboot() / _do_shutdown()</code>	สั่ง reboot/shutdown ตัวเครื่อง Raspberry Pi ผ่าน provider

7.2 carekeeper_providers.py — Provider Layer

Function/Class	หน้าที่
<code>_env(name, default)</code>	อ่านตัวแปรแวดล้อมพร้อมค่า default
<code>_subsystem_disabled(name)</code>	เช็คว่ subsystem ถูก auto-disable อยู่หรือไม่ (เชื่อมกับ <code>carekeeper_retry</code>)
<code>read_device_mac()</code>	อ่าน MAC address ของตัวเครื่อง Raspberry Pi เพื่อใส่ใน payload ที่ส่ง Backend (ข้าม lo และถอยไปใช้ <code>uuid.getnode()</code> เมื่ออ่านไม่ได้)
<code>PatientInfo</code>	dataclass เก็บข้อมูลผู้รับบริการ (เลขบัตร, ชื่อ, วันเกิด, ที่อยู่)
<code>BloodPressureReading</code>	dataclass เก็บผลวัดความดัน (SYS/DIA/PUL)
<code>DeviceStatus</code>	dataclass เก็บสถานะ Battery/Wi-Fi/Bluetooth และสถานะ disabled ของแต่ละ subsystem
<code>MeasurementHistoryRecord</code>	dataclass เก็บ 1 แถวของข้อมูลย้อนหลัง
<code>CareKeeperProvider (ABC)</code>	interface กลางที่ UI เรียกใช้ ประกาศเมธอดที่ Mock/Real ต้อง implement เช่น <code>read_patient</code> , <code>measure_blood_pressure</code> , <code>measure_spo2</code> , <code>measure_temperature</code> , <code>get_device_status</code> , <code>get_measurement_history</code> , <code>send_data</code> , <code>scan_wifi_networks</code> , <code>connect_wifi</code> , <code>scan_bluetooth_devices</code> , <code>connect_bluetooth</code> , <code>reboot_device</code> , <code>shutdown_device</code> และ <code>get_ip_address</code> (เพิ่มใหม่)
<code>MockCareKeeperProvider</code>	สร้างข้อมูลจำลองทั้งหมด สำหรับพัฒนา/ทดสอบ UI โดยไม่ต้องต่ออุปกรณ์จริง
<code>RealCareKeeperProvider</code>	เชื่อมต่ออุปกรณ์จริงผ่านโมดูลใน lib/ และ Backend จริงผ่าน requests

เมธอดหลักของ RealCareKeeperProvider (บางส่วน)

Method	หน้าที่
read_patient()	เรียก lib/thaiidcard/card.py เพื่ออ่านบัตรประชาชนจริง
measure_blood_pressure()	เรียก lib/bp_monitor.py เพื่อวัดความดัน/ซีพจรผ่าน serial; เก็บสาเหตุที่วัดไม่ได้ไว้ที่ last_bp_error (BP_ERROR / NOT_READY / TIMEOUT) เพื่อให้ UI ตัดสินใจล็อกปุ่มได้
_resolve_bp_port()	[ใหม่] หาพอร์ต serial ของเครื่องวัดความดันแบบอัตโนมัติ ลำดับความสำคัญคือ ค่าใน CAREKEEPER_BP_PORT ที่ยังเสียบอยู่ → พอร์ต USB-serial ที่รู้จักชิป (CH34x/CP210x/FTDI/Prolific/ESP32) → พอร์ต USB-serial ตัวเดียวที่เหลือ ทำให้เลข /dev/ttyUSB* เปลี่ยนได้โดยไม่ต้องแก้ .env (อ่านอย่างเดียว ไม่เปิดพอร์ตจริง)
measure_spo2() / _open_spo2_sensor() / _read_spo2()	[เปลี่ยนใหม่] วัด SpO2 จากเซนเซอร์ MAX30102 ผ่าน I2C (แทน H59 BLE เดิม) โดยแยกชั้น "เปิดเซนเซอร์" ซึ่ง retry/auto-disable ได้ ออกจากชั้น "อ่านค่า" ซึ่งจะรอนค่านิ่งเอง จึงไม่ปิด subsystem เพียงเพราะผู้ใช้อยู่ไม่วางนิ้ว
_spo2_failure_message(monitor)	[ใหม่] แปลงสาเหตุที่วัดไม่สำเร็จเป็นข้อความที่ผู้ใช้ทำตามได้ 3 แบบ (ไม่พบนิ้ว / สัญญาณอ่อน / ค่ายังไม่นิ่ง) พร้อมแนบค่า IR ที่วัดได้เทียบกับ threshold สำหรับใช้ตั้งค่า CAREKEEPER_SPO2_FINGER_IR_THRESHOLD
measure_temperature()	[เปลี่ยนใหม่] วัดอุณหภูมิร่างกายจากเซนเซอร์ DS18B20 ผ่าน lib/temp_sensor.py (เดิมเป็น NotImplementedError) โดยรอนค่านิ่งแล้วคืนค่ามัธยฐาน
_read_battery_percent()	อ่านเปอร์เซ็นต์แบตเตอรี่ผ่าน lib/ups.py; คืน None แทน 0 เมื่ออ่านไม่ได้
_is_wifi_connected() / _is_bluetooth_connected()	เช็คสถานะเชื่อมต่อผ่าน nmcli / bluetoothctl
toggle_wifi() / toggle_bluetooth()	[ใหม่] เปิด/ปิดวิทยุ Wi-Fi หรือ Bluetooth ของเครื่อง (nmcli radio wifi, bluetoothctl power)
send_data(payload)	POST payload (JSON) ไปยัง CAREKEEPER_API_URL พร้อม header API key
scan_wifi_networks() / connect_wifi(ssid, password)	สแกน/เชื่อมต่อ Wi-Fi ผ่าน nmcli (มี timeout กันค้าง, รอนรับ SSID ที่ซ่อนชื่อ และคืนโปรไฟล์เดิมกลับเมื่อเชื่อมต่อไม่สำเร็จ)
scan_bluetooth_devices() / connect_bluetooth(address)	สแกน/จับคู่อุปกรณ์ Bluetooth ผ่าน bluetoothctl และ bleak
get_ip_address()	[ใหม่] คืน IP ปัจจุบันของเครื่องจากคำสั่ง hostname -I (IPv4 ก่อน) ใช้แสดงในหน้าต่างข้อมูลเครื่อง; คืนค่าว่างเมื่อยังไม่ได้ต่อเครือข่าย
reboot_device() / shutdown_device()	สั่ง reboot/shutdown ตัวเครื่อง Raspberry Pi

7.3 carekeeper_retry.py — Reliability Layer

Class/Function	หน้าที่
SubsystemState	เก็บสถานะของ subsystem หนึ่งตัว (เช่น wifi, idcard) — จำนวนครั้งที่ fail ล่าสุด, enable/disable
SubsystemState.mark_failure(reason) / mark_success()	อัปเดตสถานะเมื่อเรียกอุปกรณ์สำเร็จ/ล้มเหลว
SubsystemState.disable(reason) / enable()	ปิด/เปิดการใช้งาน subsystem
SubsystemRegistry	singleton เก็บสถานะทุก subsystem ทั้งโปรเซส (wifi, bluetooth, idcard, bp_monitor, spo2)
retry_with_notify(...)	wrapper เรียกฟังก์ชันอุปกรณ์ซ้ำสูงสุด 3 ครั้งแบบ linear backoff; แจ้ง callback ทุกครั้งที่ retry; ถ้าครบจำนวนแล้วยังไม่สำเร็จ จะ disable() subsystem นั้น
retry_with_notify_async(...)	[ใหม่] รัน async ของ retry_with_notify สำหรับ coroutine (เช่นงาน BLE ที่ต้อง await)

7.4 carekeeper_queue.py — Offline Queue Layer

Class/Function	หน้าที่
QueuedSubmission	dataclass แทนหนึ่งแถวใน SQLite queue
SubmissionQueue	wrapper รอบ SQLite ที่ data/carekeeper_queue.db
SubmissionQueue.enqueue(payload)	บันทึก payload ที่ส่งไม่สำเร็จลงคิว
SubmissionQueue.peek_pending(limit)	ดึงรายการที่รอส่ง
SubmissionQueue.mark_sending/_failed/_sent_and_delete(...)	อัปเดตสถานะแถวระหว่าง/หลังพยายามส่ง
SubmissionQueue.count_pending()	นับจำนวนรายการค้างส่ง
SubmissionQueue.reset_stuck_sending()	รีเซ็ตแถวที่ค้างสถานะ sending (เช่น โปรแกรม ถูกปิดกลางคัน) กลับเป็น pending ตอนเริ่ม โปรแกรมใหม่
QueueDrainWorker (threading.Thread)	background daemon thread ที่ poll คิวทุก 5 วินาที และส่งข้อมูลค้างเมื่อกลับมาออนไลน์

7.5 carekeeper_logging.py — Logging

Function	หน้าที่
configure_logging(level)	ตั้งค่า logging กลางของทั้งระบบ (format, level)
log_thread_identity(context)	บันทึก thread id/name ปัจจุบัน — ใช้พิสูจน์ว่างาน hardware/network รันอยู่คนละ thread จาก UI จริง

7.6 carekeeper_style.py — Styling

Function	หน้าที่
build_stylesheet(font_family, number_font_family)	สร้าง QSS stylesheet string ทั้งหมดของแอป (สี, ขนาดฟอนต์, สถานะ hover/pressed ฯลฯ)

7.7 lib/ — Hardware Drivers

lib/thaiidcard/card.py

Function/Class	หน้าที่
format_thai_birth_date(raw)	แปลงวันเกิดรูปแบบดิบจากบัตร (พ.ศ.) เป็นข้อความอ่านง่าย
CardInfo	dataclass เก็บข้อมูลที่อ่านได้ จากบัตร
ThaiIDCard.connect()	เชื่อมต่อเครื่องอ่านบัตรผ่าน PC/SC (pyscard)
ThaiIDCard._transmit/_select_card/_read_apdu/_decode(...)	สื่อสารระดับ APDU กับชิปบัตร
ThaiIDCard.get_cid/get_th_name/get_en_name/get_birth_date/get_address()	อ่านแต่ละ field จากบัตร
ThaiIDCard.read()	อ่านข้อมูลทั้งหมดจากบัตรในครั้งเดียว

lib/bp_monitor.py

Function/Class	หน้าที่
BPResult	dataclass ผลวัดความดัน (SYS/DIA/PUL)
BPMonitor.connect() / disconnect()	เปิด/ปิดพอร์ต serial ไปยังเครื่องวัดความดัน
BPMonitor.measure(blocking)	สั่งเริ่มวัดและรอผล
BPMonitor.last_error	[ใหม่] สาเหตุที่ measure() ไม่ได้ค่า: BP_ERROR (เครื่องแจ้ง fault เอง), NOT_READY หรือ TIMEOUT — UI ใช้ค่านี้นัดเตือนใจล็อกปุ่มวัด
BPMonitor._read_loop() / _handle_line() / _parse_result()	อ่าน/แปลงข้อมูลดิบจาก serial เป็น BPResult

lib/temp_sensor.py [ใหม่ — เซนเซอร์อุณหภูมิ DS18B20 ผ่าน 1-Wire]

Function/Class	หน้าที่
temp_sensor(sensor_id, calibration_offset, ...)	หาเซนเซอร์จาก /sys/bus/w1/devices/28-* อัตโนมัติ และตั้งค่าชดเชยอุณหภูมิ (calibration offset) กับเงื่อนไขการหาตัวนี้
read_celsius_once()	อ่านค่าดิบหนึ่งครั้งจากไฟล์ w1_slave พร้อมตรวจ CRC (ลองใหม่ได้สูงสุด 5 ครั้ง)
is_probe_in_contact(temp)	ตรวจว่าเซนเซอร์แนบผิวจริงหรือไม่ โดยดูว่าค่าอยู่ในช่วง 34–43 °C
measure_body_temperature(on_progress)	วนอ่านค่าจนกว่า 5 ค่าล่าสุดจะต่างกันไม่เกิน 0.05 °C แล้วคืนค่ามัธยฐาน (สูงสุด 60 วินาที) ถ้าเซนเซอร์หลุดจากผิวระหว่างวัด จะล้างค่าที่สะสมไว้แล้วเริ่มนับใหม่
get_id() / list_sensors()	ดู ID ของเซนเซอร์ที่ใช้อยู่ และรายชื่อเซนเซอร์ 1-Wire ทั้งหมดที่ต่ออยู่

lib/spo2_max30102/ [ใหม่ — เซนเซอร์ SpO2 MAX30102 ผ่าน I2C]

Function/Class	ไฟล์	หน้าที่
MAX30102	max30102.py	ตั้งค่าและอ่าน FIFO ของชิปผ่าน smbus/smbus2 (สุ่มสัญญาณ 100 Hz เฉลี่ย 4 ตัวอย่าง เหลืออัตราออก 25 Hz) พร้อมตัวนับ FIFO overflow
calc_hr_and_spo2()	hrcalc.py	อัลกอริทึมคำนวณชีพจรและ SpO2 จากหน้าต่างสัญญาณ 100 ตัวอย่าง (~4 วินาที) ใช้ตามต้นฉบับของผู้ผลิตเซนเซอร์ ไม่ได้แก้ไข
SpO2Monitor.GetSpO2Sensor()	spo2_monitor.py	อ่านค่าดิบหนึ่งครั้ง (ค่ากระโดดไปมาระหว่างครั้ง) ใช้สำหรับ debug/ตรวจสอบฮาร์ดแวร์

SpO2Monitor.is_finger_present(ir)	spo2_monitor.py	ตรวจสอบว่ามีนิ้ววางอยู่จริงจากระดับ IR เฉลี่ย เทียบกับ FINGER_IR_THRESHOLD (ตั้งค่าได้ผ่าน .env) — เทียบเท่ากับการตรวจการแนบนิ้วของเซนเซอร์อุณหภูมิ
SpO2Monitor.measure_spo2(on_progress)	spo2_monitor.py	เลื่อนหน้าต่างสัญญาณทีละ 1 วินาที คำนวณค่าใหม่ทุกครั้ง ทั้งค่าที่เป็นไปไม่ได้ (นอกช่วง 70–100%) และทั้งหน้าต่างที่ FIFO ขาดช่วงจนกว่า 5 ค่าติดกันจะต่างกันไม่เกิน 3% แล้วคืนค่ามัธยฐาน (สูงสุด 60 วินาที) ถ้ายกนิ้วออกกลางคันจะล้างค่าและเริ่มใหม่
SpO2Monitor.last_error	spo2_monitor.py	สาเหตุที่วัดไม่สำเร็จ: NO_FINGER / WEAK_SIGNAL / UNSTABLE

lib/h59_ble/ [เดิม — ไม่ได้ใช้เป็นแหล่ง SpO2 แล้ว เก็บไว้อ้างอิง]

Function/Class	ไฟล์	หน้าที่
H59Device	device.py	เชื่อมต่อ BLE กับอุปกรณ์ H59, ส่ง/รับค่า สั่ง, keepalive connection
SpO2Reader	spo2.py	รับ BLE notification แล้ว parse เป็นค่า SpO2
HeartRateReader	heart_rate.py	รับ BLE notification แล้ว parse เป็นค่า ชีพจร

lib/ups.py

Function/Class	หน้าที่
UPSHat	เชื่อมต่อ UPS HAT ผ่าน I2C
get_battery_percent()/get_battery_voltage()/get_vbus_* ()	อ่านค่าพลังงาน/แบตเตอรี่ต่างๆ
get_all()	คืนค่าทุก field พร้อมกันในครั้งเดียว
shutdown()	สั่งปิดเครื่องผ่าน UPS HAT

8. การจัดการ Thread และ Concurrency

Python มี Global Interpreter Lock (GIL) ทำให้ multi-thread ไม่ได้ประโยชน์เต็มที่กับงาน CPU-bound แต่ งานส่วนใหญ่ของ CareKeeper เป็น I/O-bound (รอ serial, BLE, smart card, HTTP) จึงเหมาะกับการใช้ thread

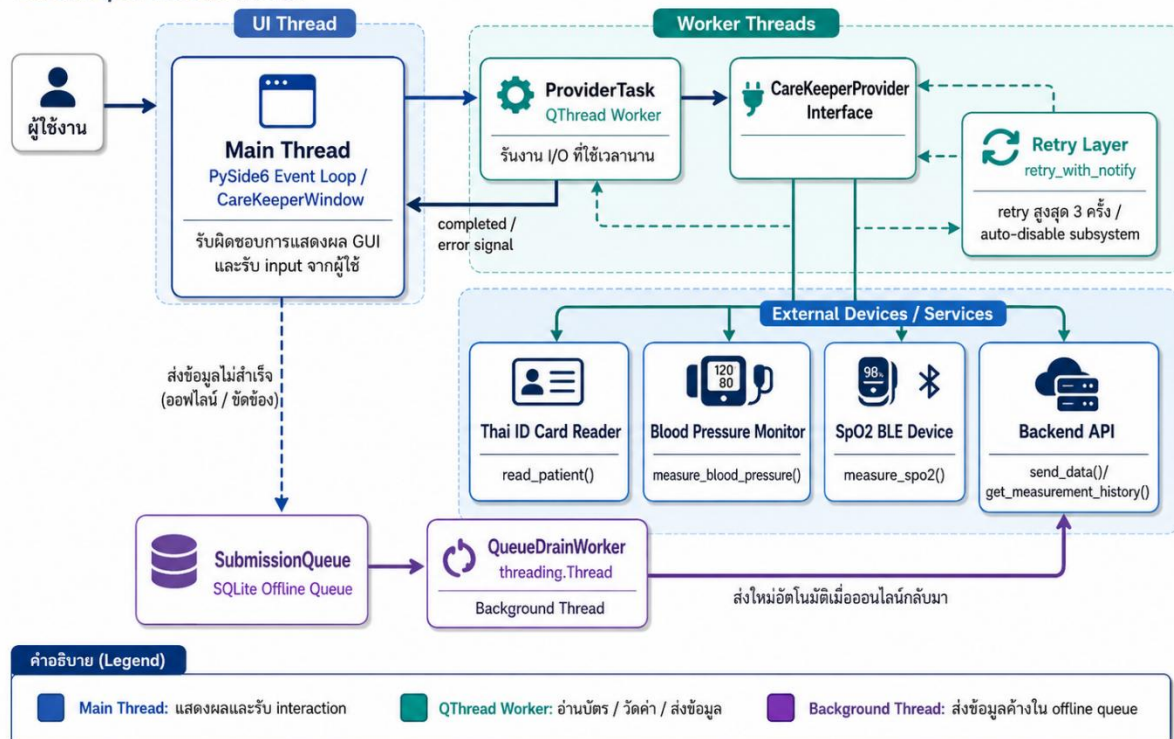
หลักการ

- Main thread (PySide6 event loop) รับผิดชอบเฉพาะการวาดผลและรับ interaction ของผู้ใช้
- งานที่รออุปกรณ์/network ถูกส่งเข้า ProviderTask (QThread) เสมอ ได้แก่ อ่านบัตร, วัดความดัน, วัด SpO2 (รอจนค่านิ่ง สูงสุด 60 วินาที), วัดอุณหภูมิ (สูงสุด 60 วินาที), ส่งข้อมูลไป Backend, สแกน/เชื่อมต่อ Wi-Fi และ Bluetooth, อ่านสถานะอุปกรณ์เป็นรอบ
- QueueDrainWorker เป็น threading.Thread แยกต่างหาก คอยส่งข้อมูลค้างใน offline queue แบบ background ตลอดอายุโปรแกรม
- คำสั่งระบบ (nmcli, bluetoothctl, iwgetid, ip link, hostname -l) ถูกกำหนด timeout ทุกคำสั่ง เพื่อกัน subprocess ค้างบน Raspberry Pi
- มี work_task guard กันผู้ใช้กดสแกน/เชื่อมต่อ Wi-Fi หรือ Bluetooth ซ้ำกันหลายครั้ง

8.1 แผนภาพ Thread Model (Template)

8.1 แผนภาพ Thread Model

CareKeeper Thread Model



คำอธิบายได้ภาพ

ระบบแยก thread ออกเป็น 3 กลุ่มหลัก ได้แก่

1. Main Thread

รับผิดชอบการแสดงผล GUI, รับ input จากผู้ใช้ และอัปเดตสถานะบนหน้าจอเท่านั้น

2. ProviderTask / QThread

ใช้รันงานที่ใช้เวลานานหรือรอ I/O เช่น อ่านบัตรประชาชน วัดความดัน วัด SpO2 วัดอุณหภูมิ ส่งข้อมูลไป Backend และสแกน/เชื่อมต่อ Wi-Fi หรือ Bluetooth

3. QueueDrainWorker / Background Thread

ใช้ส่งข้อมูลที่บันทึกค้างไว้ใน offline queue เมื่อระบบส่ง Backend ไม่สำเร็จ เช่น อินเทอร์เน็ตหลุด หรือ server ไม่ตอบกลับ

8.2 ตัวอย่าง Log พิสูจน์ (test_threading.py)

ไฟล์ tests/test_threading.py มี 4 ฟังก์ชัน ทั้งหมดพิสูจน์เรื่องเดียวกัน คือ งานหนัก/งานที่รอนาน ต้องรันบน worker thread ไม่ใช่ main (GUI) thread เพื่อไม่ให้ UI ค้าง แต่ละอันทำดังนี้

1. test_provider_task_runs_off_main_thread

สร้าง ProviderTask ให้รันฟังก์ชัน action() ที่แอบบันทึก thread id ของตัวเองไว้ รอ signal completed แล้ว เช็คว่า thread id ที่รันจริง ≠ main thread

➔ พิสูจน์ว่า งานของ provider (เช่นคุยกับอุปกรณ์/ส่งข้อมูล) รันแยก thread จาก GUI

2. test_queue_drain_worker_runs_off_main_thread

สตาร์ท QueueDrainWorker (ตัวที่ค่อย ๆ ดึงคิวไปส่ง) ด้วย send_fn ปลอมที่คืน True เสมอ เช็คว่า worker is_alive() จริง และ ident ของมัน ≠ main thread แล้วปิดให้เรียบร้อยใน finally

➔ พิสูจน์ว่า ตัว drain คิวทำงานเป็น background thread

3. test_retry_sleep_does_not_block_main_thread_log_assertion

เรียก retry_with_notify(...) จากใน worker thread ที่สร้างเอง อ่าน log ของ logger carekeeper.threads แล้ว ยืนยันว่ามีข้อความ is_main_thread=False

➔ พิสูจน์ว่า loop ที่ sleep ระหว่าง retry ไม่ได้ไปนั่งรอนบน GUI thread (ไม่ทำให้จ่อค้างตอน retry)

4. test_log_thread_identity_format

เรียก log_thread_identity("unit_test_context") แล้วตรวจ รูปแบบข้อความ log ว่ามีครบทุก field: context=, py_name=, py_id=, qt_thread=, is_main_thread=

➔ เป็น unit test ของ ตัวช่วย logging เอง

(ฟังก์ชัน 1-3 ฟังพา log นี้ในการตรวจ thread จึงต้องมั่นใจว่า format ถูก)

ตัวอย่างการรัน Test

```
===== python -m pytest tests/test_threading.py -v =====
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- /home/cpe05/Care_Keeper/carekeeper/bin/python
cachedir: .pytest_cache
PySide6 6.11.1 -- Qt runtime 6.11.1 -- Qt compiled 6.11.1
rootdir: /home/cpe05/Care_Keeper
plugins: qt-4.5.0, mock-3.15.1, cov-7.1.0
collected 4 items

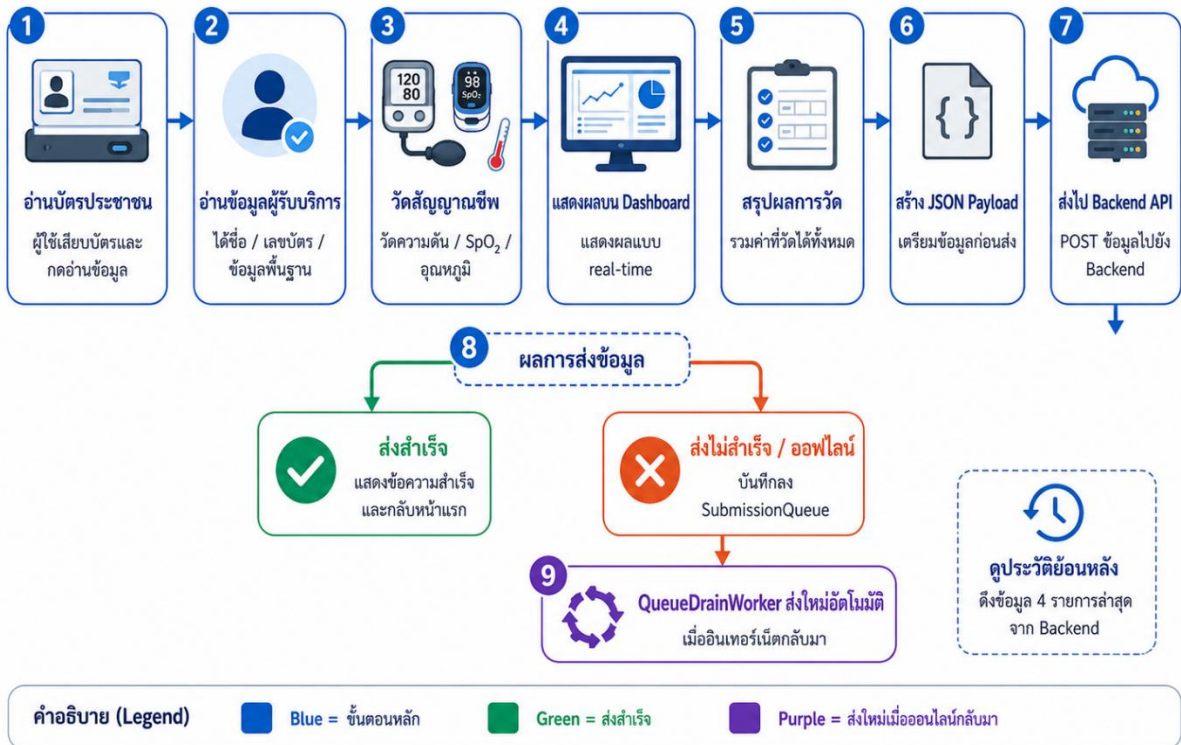
tests/test_threading.py::test_provider_task_runs_off_main_thread PASSED [ 25%]
tests/test_threading.py::test_queue_drain_worker_runs_off_main_thread PASSED [ 50%]
tests/test_threading.py::test_retry_sleep_does_not_block_main_thread_log_assertion PASSED [ 75%]
tests/test_threading.py::test_log_thread_identity_format PASSED [100%]

===== 4 passed in 0.20s =====
```

9. Data Flow: จากบัตรประชาชนถึง Backend

9.1 ลำดับการทำงานหลัก

CareKeeper Main Workflow



ลำดับขั้นตอนแบบละเอียด

- ผู้ใช้เสียบบัตรประชาชน กดปุ่ม “อ่านข้อมูลบัตร” → `CareKeeperWindow._read_card()`
→ `_start_task(provider.read_patient)`
- ProviderTask รันใน background thread เรียก `RealCareKeeperProvider.read_patient()` → `lib/thaaidcard/card.py:ThailDCard.read()` → คืน PatientInfo
 - ถ้าอ่านไม่สำเร็จ → `retry_with_notify` ลองใหม่สูงสุด 3 ครั้ง → ถ้ายังไม่สำเร็จ auto-disable + แสดง popup กรอกเลขบัตรเอง
- เข้าหน้า Dashboard → ผู้ใช้กดปุ่มวัดแต่ละชนิด → แต่ละปุ่มเรียก `_start_task` กับ provider method ที่สอดคล้อง (`measure_blood_pressure`, `measure_spo2`, `measure_temperature`)
- ค่าที่วัดได้ถูกเก็บใน VitalState แล้วอัปเดต widget แสดงผลแบบ real-time (แยกสีตามชนิดข้อมูล)
- ผู้ใช้กดดูสรุปผล → เข้าหน้า Summary → แสดงค่าทั้งหมดในตาราง
- ผู้ใช้กด “บันทึกข้อมูล” → `_submit_data()` ประกอบ JSON payload (รวม mac จาก `read_device_mac()`) → เรียก `provider.send_data(payload)`
- ถ้าส่งสำเร็จ: แสดง toast สำเร็จ → reset session กลับหน้าแรก
- ถ้าส่งไม่สำเร็จ (เช่น อินเทอร์เน็ตหลุด): payload ถูก `SubmissionQueue.enqueue()` ลง SQLite → `QueueDrainWorker` จะพยายามส่งใหม่ทุก 5 วินาทีเมื่อออนไลน์กลับมา
- (ตามต้องการ) ผู้ใช้กดดู History → เรียก `provider.get_measurement_history()` แสดง 4 รายการล่าสุด

JSON Payload ที่ส่งไป Backend

```
{
  "mac": "1c:ce:51:9a:34:77",
  "patient_id": "1234567890123",
  "spo2": 98,
  "heart_rate": 70,
  "pr_bpm": 70,
  "sys": 120,
  "dia": 78,
  "pulse": 70,
  "temperature": 36.6
}
```

JSON / Request สำหรับดึงข้อมูลประวัติจาก Backend

รูปแบบ Request

```
GET /api/v2/device/health_history?patient_id=1234567890123&mac=1c:ce:51:9a:34:77&limit=4
Content-Type: application/json
api-key: test
```

ตัวอย่าง JSON Response จาก Backend

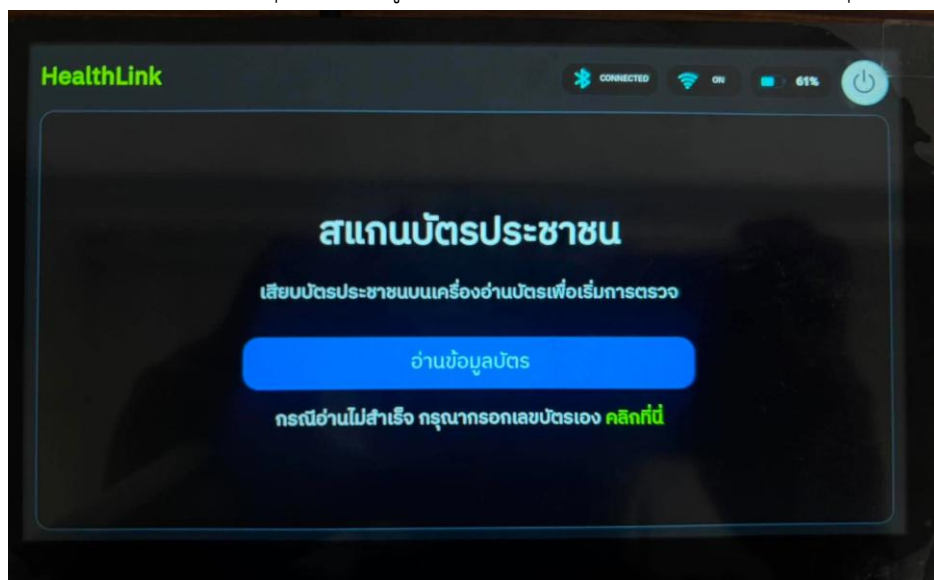
```
{
  "data": [
    {
      "measured_at": "2026-06-29T13:30:00+07:00",
      "sys": 120,
      "dia": 78,
      "pulse": 70,
      "spo2": 98,
      "temperature": 36.6
    },
    {
      "measured_at": "2026-06-29T12:40:00+07:00",
      "sys": 128,
      "dia": 82,
      "pulse": 76,
      "spo2": 97,
      "temperature": 36.7
    }
  ]
}
```

10. หน้าจอของระบบ (UI Screens) Template รูป

ระบบแบ่งหน้าจอหลักออกเป็น 3 หน้า บวกหน้าต่างเสริม (popup/selector) ด้านล่างนี้คือ template สำหรับใส่ screenshot จริงภายหลัง — ทุกภาพขนาดอ้างอิง 1010×503 px

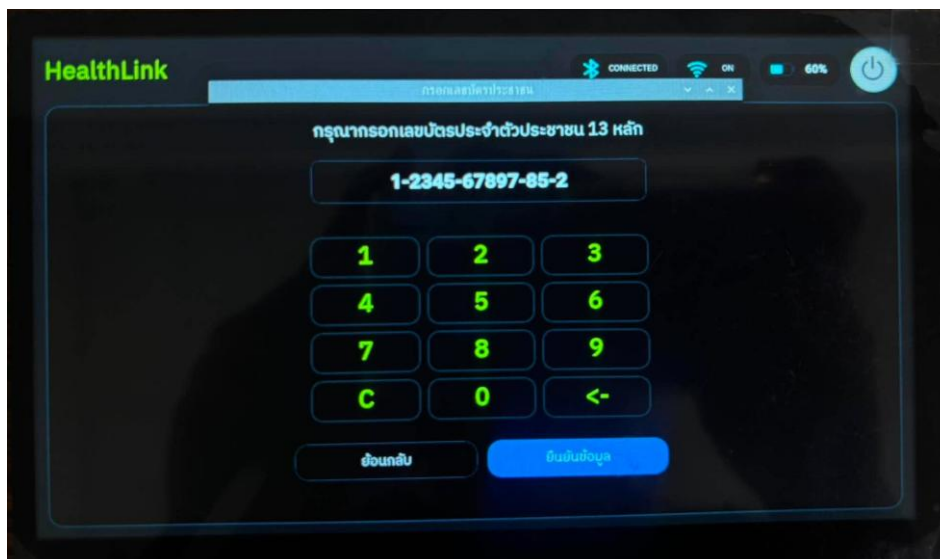
10.1 หน้าอ่านบัตรประชาชน (Scan Page)

หน้าแรกของระบบ ใช้เริ่มต้นการตรวจ มีปุ่ม “อ่านข้อมูลบัตร” และลิงก์สำรอง “กรณีอ่านไม่สำเร็จ กรุณากรอกเลขบัตรเอง”

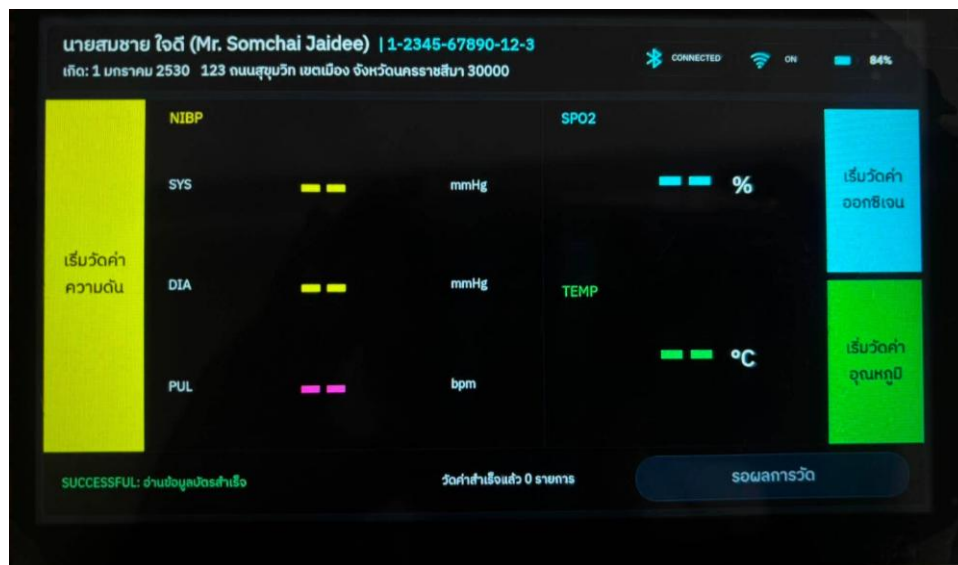


10.2 Popup กรอกเลขบัตรประชาชนด้วยตนเอง

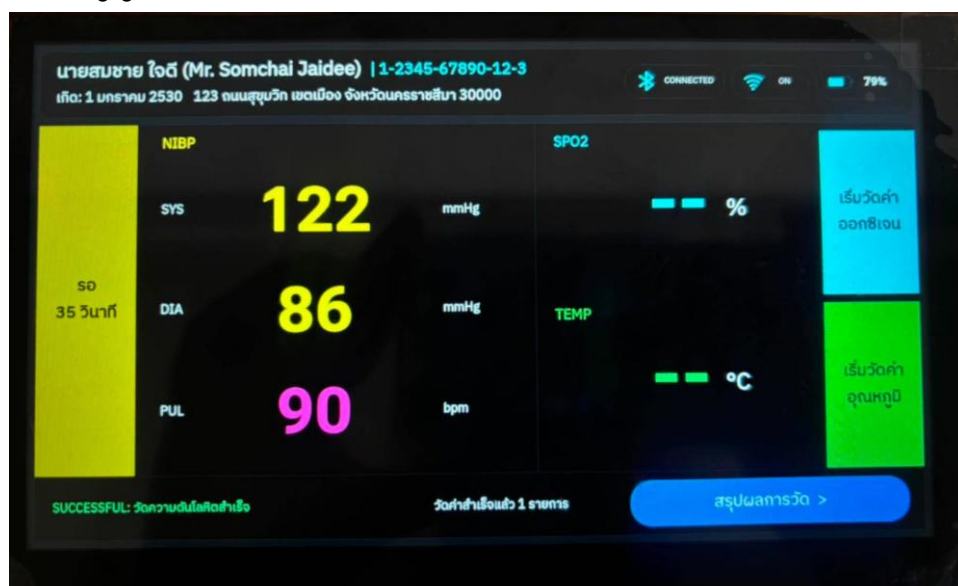
แสดงเมื่อกดลิงก์สำรอง มี on-screen numpad บังคับกรอกเฉพาะตัวเลข 0-9 จำนวน 13 หลัก



10.3 หน้าวัดค่าสัญญาณชีพ



10.4 หน้าวัดค่าสัญญาณชีพ — ผลความดันโลหิต



10.5 หน้าวัดค่าสัญญาณชีพ — ผล SpO2



10.6 หน้าวัดค่าสัญญาณชีพ — ผลอุณหภูมิ

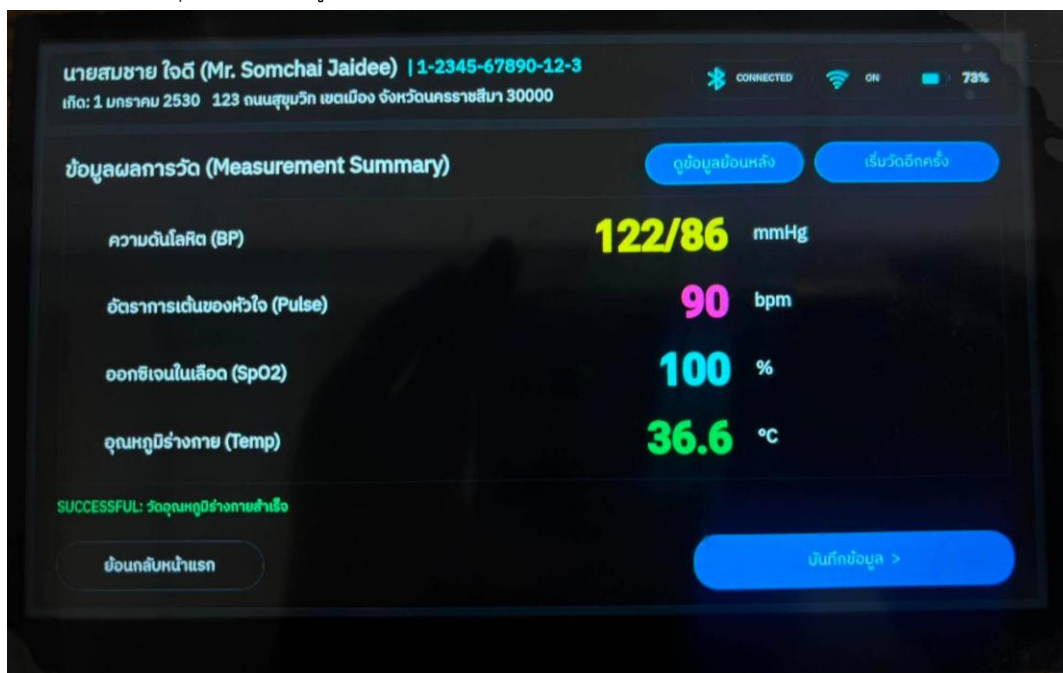


10.7 หน้าวัดค่าสัญญาณชีพทั้งหมด (All Measurement)



10.8 หน้าสรุปผลการวัด (Measurement Summary)

รวมผลการวัดทั้งหมด มีปุ่ม “บันทึกข้อมูล” ส่ง JSON ไป Backend



10.9 หน้าต่างแสดงข้อมูลย้อนหลัง (History)

แสดง 4 รายการล่าสุดในรูปแบบตาราง (วันที่/เวลา, BP, Pulse, SpO2, Temp)



นายสมชาย ใจดี (Mr. Somchai Jaidee) | 1-2345-67890-12-3
เกิด: 1 มกราคม 2530 123 ถนนสุขุมวิท เขตเมือง จังหวัดนครราชสีมา 30000

CONNECTED ON 71%

ข้อมูลผลการวัด (Measurement Summary)

สรุปผลการวัด เริ่มวัดอีกครั้ง

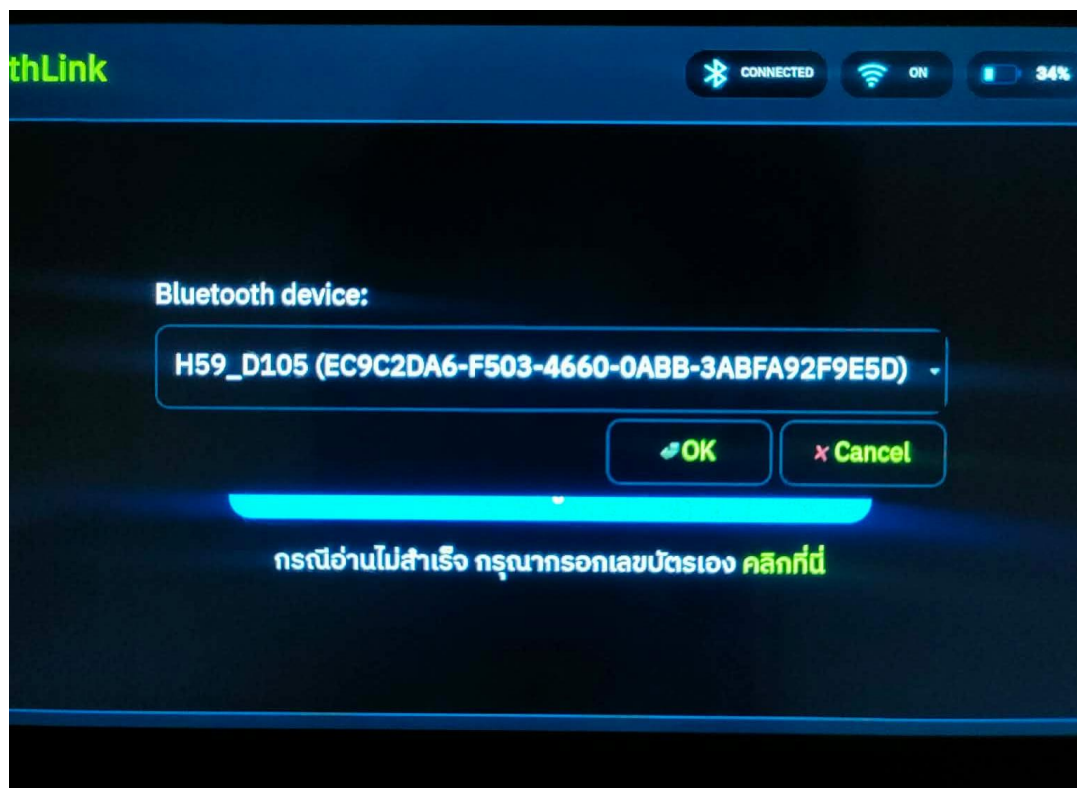
วันที่และเวลา	ความดันโลหิต (mmHg)	ชีพจร (bpm)	ออกซิเจนในเลือด (SpO2 %)	อุณหภูมิ (°C)
24/06/69 12:00	120/78	70	98	35.5
24/06/69 11:55	128/89	85	-	36.6
24/06/69 11:49	119/78	80	78	35.5
24/06/69 11:35	120/78	-	76	37.0

SUCCESSFUL: วัดอุณหภูมิร่างกายสำเร็จ

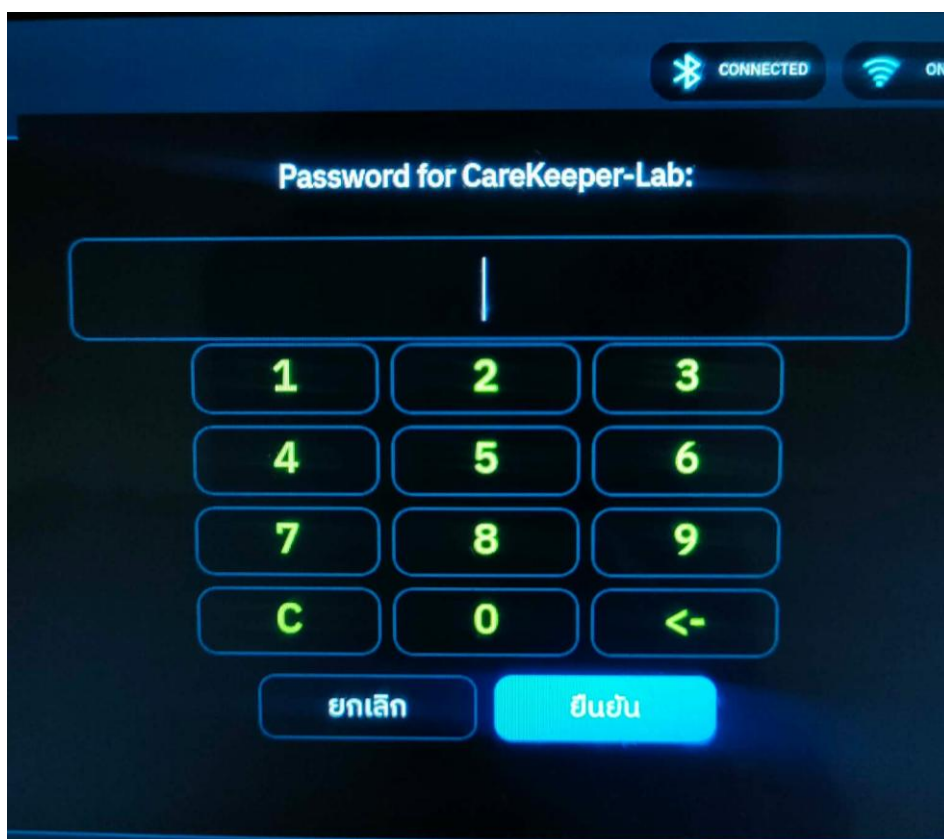
ย้อนกลับหน้าแรก บันทึกข้อมูล >

10.10 หน้าต่างเลือก Wi-Fi / Bluetooth

Bluetooth Selector Placeholder



Wi-Fi Selector Placeholder



10.11 หน้าต่างข้อมูลเครื่อง (Device ID) และปุ่มปิดหน้าจอ

เปิดจากเมนูปุ่ม Power บนแถบสถานะ — หน้าต่าง "ข้อมูลเครื่อง (Device ID)" แสดง IP, Hostname, MAC และคำสั่ง ssh ที่พร้อมคัดลอกไปใช้ ทำให้หาเครื่องเจอเพื่อ SSH/VNC เข้าไปได้แม้เพียงย้ายเครือข่าย ส่วนเมนู "ปิดหน้าจอ" จะถามยืนยันก่อนออกจากแอปกลับสู่เดสก์ท็อป โดย systemd จะไม่เปิดแอปขึ้นมาใหม่ จนกว่าจะรีบูตหรือสั่งเปิดเอง

11. การติดตั้งและรันโปรแกรม (Setup & Run)

11.1 สร้าง Python Virtual Environment

```
python -m venv .venv
source .venv/bin/activate      # Linux/Raspberry Pi
# .\.venv\Scripts\Activate.ps1 # Windows PowerShell

pip install -r requirement.txt
# สำหรับ dev/test เพิ่ม:
pip install -r requirements-dev.txt
```

11.2 System packages (Raspberry Pi)

```
sudo apt update
sudo apt install python3-smbus i2c-tools bluetooth bluez pcscd libpcsclite-dev swig python3-dev
sudo usermod -aG dialout,bluetooth,i2c $USER
sudo reboot
```

11.3 ตั้งค่า .env

```
cp .env.example .env
# แก้ค่าตาม Backend/อุปกรณ์จริง แล้วห้าม commit ไฟล์นี้
```

11.4 รันโปรแกรม

```
python main_demo.py      # Mock UI Mode – ทดสอบหน้าจอโดยไม่ต้องต่ออุปกรณ์จริง
python main_real.py      # Real Hardware Mode – รันบน Raspberry Pi ที่ต่ออุปกรณ์จริง
```

11.5 ทดสอบอุปกรณ์แยกเดี่ยว

```
python sensor_tests/idcard.py      # ทดสอบเครื่องอ่านบัตร
python sensor_tests/BP.py          # ทดสอบเครื่องวัดความดัน
python sensor_tests/H59_BLE.py     # ทดสอบอุปกรณ์ SpO2/Heart-rate
python sensor_tests/battery.py     # ทดสอบ UPS/แบตเตอรี่
python sensor_tests/ble_scanner.py # ทดสอบค้นหา Bluetooth device
```

12. การ Deploy บน Raspberry Pi 5 (Kiosk Mode)

งาน	รายละเอียด	ไฟล์อ้างอิง
Auto-run ตอนบูต	systemd service สั่งรัน main_real.py ด้วย venv ของ CareKeeper ทันทีที่เข้า graphical target, รอ pcsd พร้อมก่อน, Restart=on failure	conf/carekeeper.service
Restart ทุกคืน	timer สั่ง reboot ทุกวันที 3 เพื่อ ล้าง memory/cache	conf/carekeeper-restart.service, .timer
จำกัดขนาด log	systemd-journald ใช้พื้นที่ไม่ เกิน 200MB เก็บย้อนหลังไม่เกิน 14 วัน	conf/carekeeper-journal.conf
สิทธิ์ USB เครื่องอ่านบัตร/sensor	udev rule ให้ USB เข้าถึงได้โดย ไม่ต้องเป็น root	/etc/udev/rules.d/99-thaيدcard.rules (ทำ บนเครื่อง)
อ่านบัตรได้แม้ไม่มีคน login	polkit rule ให้ pcsd อนุญาต client ที่ไม่มี active session	/etc/polkit-1/rules.d/50-pcsd.rules (ทำ บนเครื่อง)
ซ่อน desktop/taskbar	แก้ autostart ของ labwc (Wayland compositor)	/etc/xdg/labwc/autostart, ~/.config/labwc/autostart
ซ่อนข้อความ “Welcome to...” ตอนบูต	แทนที่ splash.png ของ Plymouth ด้วยพื้นสีเดียว	/usr/share/plymouth/themes/pix/splash.png (มี .bak)
ปิดหน้าจอแล้วไม่ดับกลับ	[ใหม่] แอปออกด้วย exit code 42 เมื่อผู้ใช้สั่ง "ปิดหน้าจอ" และ systemd ตั้ง RestartPreventExitStatus=42 ไว้ จึงไม่เปิดแอปซ้ำ ส่วนการ crash จริง (exit code อื่น) ยังกู้คืนอัตโนมัติเหมือนเดิม	conf/carekeeper.service
เปิด I2C และ 1-Wire	[ใหม่] เปิด I2C ใน raspi-config สำหรับ UPS HAT และ เซนเซอร์ SpO2 MAX30102 และเพิ่ม dtoverlay=w1-gpio สำหรับเซนเซอร์อุณหภูมิ DS18B20 (ตรวจด้วย i2cdetect -y 1 และ ls /sys/bus/w1/devices/)	/boot/firmware/config.txt (ทำบนเครื่อง)

ลำดับ Phase ใน Runbook

ลำดับ Phase ใน Runbook

Raspberry Pi Kiosk Setup Workflow



13. การทดสอบ (Testing)

โปรเจกต์มีชุด pytest ครอบคลุมหลายด้าน อยู่ในโฟลเดอร์ tests/ ปัจจุบัน 231 test cases ผ่านทั้งหมด (100%) และวัด code coverage ของโมดูล logic (ไม่รวม carekeeper_ui.py) ได้ 70% โดยตัวหารเพิ่มขึ้นจากเดิมเพราะรวมไลบรารีเซนเซอร์ที่เพิ่มเข้ามาใหม่ (lib/spo2_max30102/ และ lib/temp_sensor.py) ซึ่งส่วนใหญ่เป็นโค้ดคุยกับฮาร์ดแวร์โดยตรง รายละเอียดเต็มอยู่ใน docs/test-report.md

ไฟล์ทดสอบ	ครอบคลุม	จำนวน
test_providers_bp.py	การวัดความดันของ provider: retry, รหัสข้อผิดพลาด และ auto-detect พอร์ต	11
test_providers_spo2.py	การวัด SpO2 ระดับ provider: retry ตอนเปิดเซนเซอร์, timeout, ข้อความบอกสาเหตุ	13
test_spo2_stability.py	[ใหม่] การหาค่านิ่งของ SpO2 (MAX30102): ตรวจสอบ, ทิ้งค่าที่เป็นไปไม่ได้, คืนค่ามัธยฐาน	23
test_providers_idcard.py	การอ่านบัตรประชาชน (ใช้ fake smart card)	2
test_providers_history.py	การดึงข้อมูลย้อนหลัง (GET health_history)	9
test_providers_send_data.py	การส่งผลวัดขึ้น backend (POST add_health)	8
test_providers_wifi_bt.py	การสแกน/เชื่อมต่อ Wi-Fi และ Bluetooth	15
test_queue.py	พฤติกรรม SubmissionQueue (enqueue/dequeue/reset stuck)	7
test_queue_worker.py	พฤติกรรม QueueDrainWorker แบบ background	5
test_retry_helper.py	retry_with_notify และ SubsystemRegistry (auto disable/enable)	8
test_threading.py	ยืนยันว่างาน hardware/network รันคนละ thread จาก UI	4
test_ui_status_disabled.py	UI แสดงสถานะถูกต้องเมื่อ subsystem ถูก disable	6
test_ui_submit_flow.py	flow กดบันทึกข้อมูลจนถึงส่ง/เข้า offline queue	3
test_ui_bp_cooldown.py	[ใหม่] การล็อกปุ่มวัดความดันและนับถอยหลังหลังวัดสำเร็จ/เครื่องแจ้ง error	9
test_ui_elided_label.py	[ใหม่] การย่อข้อความยาวด้วยจุดไข่ปลาในแถบข้อมูลผู้รับบริการ	2
test_device_status_fields.py	ความถูกต้องของ field สถานะอุปกรณ์ (battery/wifi/bluetooth)	1
test_h59_packets.py	โครงสร้างแพ็กเก็ต BLE 16 ไบต์ของ H59 และ checksum	7
test_h59_notify_parsers.py	ขอบเขตค่าที่ parser ยอมรับ: ชีพจร 30–220 bpm, SpO2 70–100%	23

test_h59_reader_flows.py	ลำดับคำสั่งวัดค่าเต็ม (warm-up → start → stop/disconnect)	12
test_h59_device.py	ชั้นสื่อสาร BLE: routing UART1/2, keepalive, notify fan-out	9
test_thaiidcard.py	แปลงวันเกิด พ.ศ., decode TIS-620, โปรโตคอล APDU (GET RESPONSE)	15
test_ups.py	ถอดรหัส register I2C ของ UPS HAT (แรงดัน/กระแส/% แบตเตอรี่)	15
test_bp_monitor_parsing.py	parser โปรโตคอล serial ของเครื่องวัดความดัน (SYS:.../DIA:.../PUL:...)	24
รวม		231

ผลลัพธ์การรัน TEST CASES ทั้งหมด

```

===== tests coverage =====
_____ coverage: platform linux, python 3.13.5-final-0 _____

Name                               Stmts  Miss  Cover
-----
carekeeper_logging.py              12      1   92%
carekeeper_providers.py           358     130   64%
carekeeper_queue.py               106      5   95%
carekeeper_retry.py               109      4   96%
lib/bp_monitor.py                 120     15   88%
lib/h59_ble/__init__.py             5       0  100%
lib/h59_ble/device.py              119     44   63%
lib/h59_ble/heart_rate.py          75      3   96%
lib/h59_ble/spo2.py                57      1   98%
lib/thaiidcard/apdu.py              6       0  100%
lib/thaiidcard/card.py             67     18   73%
lib/ups.py                         81     15   81%
-----
TOTAL                             1115    236   79%
===== 160 passed, 115 warnings in 2.99s =====

```

14. การจัดการข้อผิดพลาดและการแจ้งเตือน

กรณี	วิธีแจ้งเตือนผู้ใช้
อ่านบัตรไม่สำเร็จ	เปิด popup กรอกละบบบัตรเอง พร้อมข้อความผิดพลาด
กรอกละบบบัตรไม่ครบ/ไม่ใช่ตัวเลข	แสดงข้อความในหน้าต่าง popup และ toast แจ้งเตือน
วัดความดันไม่สำเร็จ	ปุ่มเปลี่ยนเป็นสถานะ "วัดไม่สำเร็จ" พร้อม system message; ถ้าเครื่องแจ้ง BP_ERROR (เครื่องล็อกตัวเอง) ปุ่มจะถูกล็อกจนถอยหลัง 60 วินาที กันการกดซ้ำใส่เครื่องที่ยังไม่พร้อม
วัด SpO2 ไม่สำเร็จ	[ใหม่] แยกข้อความตามสาเหตุ 3 แบบ: ไม่พบนิ้วบนเซนเซอร์ (พร้อมค่า IR ที่วัดได้), สัญญาณอ่อนเกินไป, หรือค่ายังไม่นิ่ง เพื่อให้ผู้ใช้รู้ว่าต้องทำอะไร
วัดอุณหภูมิไม่สำเร็จ	[ใหม่] แจ้งว่าไม่พบเซนเซอร์ DS18B20 หรือค่ายังไม่นิ่ง (ให้แนบเซนเซอร์กับผิวแล้วรอ)
ส่งข้อมูล Backend ไม่สำเร็จ	แจ้ง error + เก็บ payload ไว้ใน offline queue เพื่อ retry อัตโนมัติ
โหลดข้อมูลย้อนหลังไม่สำเร็จ	แสดงข้อความในตารางประวัติ
Wi-Fi/Bluetooth เชื่อมต่อไม่สำเร็จ	toast แจ้งเตือน
อ่านสถานะ Wi-Fi/Bluetooth/แบตเตอรี่ไม่ได้	toast แบบจำกัดความถี่ (ไม่รบกวนซ้ำ)
ส่งข้อมูลค้างใน offline queue ไม่สำเร็จ	toast แบบจำกัดความถี่ + worker retry ต่อเนื่อง
อ่านค่าแบตเตอรี่ไม่ได้	แสดง --% แทน 0% เพื่อไม่ให้เข้าใจผิดว่าแบตเตอรี่หมด
ไม่พบพอร์ตเครื่องวัดความดัน	[ใหม่] แจ้งว่าไม่มีอุปกรณ์ USB serial เสียบอยู่ (แทนที่จะ error พอร์ตผิดแบบเดิม)

15. ภาคผนวก (Appendix)

15.1 ตารางค่าที่ระบบแสดงผล

รายการ	รายละเอียด	หน่วย
ความดันโลหิต	SYS/DIA	mmHg
ชีพจร	Pulse	bpm
ออกซิเจนในเลือด	SpO2	%
อุณหภูมิร่างกาย	Temp	°C
ข้อมูลย้อนหลัง	4 รายการล่าสุด	วันที่/เวลา + ค่าที่วัดได้
สถานะอุปกรณ์	Wi-Fi, Bluetooth, Battery	สถานะ/เปอร์เซ็นต์

15.2 สถานะการเชื่อมต่ออุปกรณ์จริง (สรุป)

ส่วนของระบบ	สถานะ	โมดูลที่เกี่ยวข้อง
อ่านบัตรประชาชน	เชื่อมต่อจริงแล้ว	lib/thaiidcard/card.py
วัดความดัน/ชีพจร	เชื่อมต่อจริงแล้ว + auto-detect พอร์ต	lib/bp_monitor.py
วัด SpO2	เชื่อมต่อจริงผ่าน I2C แล้ว (MAX30102)	lib/spo2_max30102/
วัดอุณหภูมิ	เชื่อมต่อจริงผ่าน 1-Wire แล้ว (DS18B20)	lib/temp_sensor.py
แบตเตอรี่	เชื่อมต่อจริงแล้ว	lib/ups.py
Wi-Fi	ใช้ nmcli	Raspberry Pi/Linux
Bluetooth	ใช้ bluetoothctl / bleak	Raspberry Pi/Linux
แสดง IP / Device ID	ใช้ hostname -I	get_ip_address()
ส่งข้อมูล Backend	ส่งผ่าน HTTP POST แล้ว	requests.post()
ข้อมูลย้อนหลัง (real)	รอ GET API จาก Backend	get_measurement_history()
SpO2 ผ่าน Bluetooth (H59)	เลิกใช้แล้ว เก็บโค้ดไว้อ้างอิง	lib/h59_ble/

16. ข้อจำกัดปัจจุบันและงานที่ควรทำต่อ (Known Limitations / Roadmap)

1. ข้อมูลย้อนหลังจริงยังไม่มี GET API — mock ใช้งานได้แล้ว แต่ real mode ยังรอ endpoint จาก Backend
2. ค่าที่วัดได้ยังไม่ผ่านการสอบเทียบทางการแพทย์ — `hrcalc.py` ใช้อัลกอริทึมของผู้ผลิตเซนเซอร์ตามต้นฉบับ และ `temp_sensor.py` ใช้ค่าชดเชย (`calibration_offset`) แบบคงที่ ควรเทียบกับเครื่องมือมาตรฐานก่อนใช้จริง
3. ต้องปรับ `CAREKEEPER_SPO2_FINGER_IR_THRESHOLD` ต่อเครื่อง — ระดับ IR ของบอร์ด MAX30102 แต่ละตัวไม่เท่ากัน ถ้าตั้งสูงเกินไปจะตรวจไม่พบนิ้วจริง (ดูค่าจริงด้วย `python -m lib.spo2_max30102.spo2_monitor --raw`)
4. ไดรเวอร์ H59 BLE ยังอยู่ในโค้ดและชุดทดสอบ (51 test cases) แต่ไม่ได้ใช้เป็นแหล่ง SpO2 แล้ว ควรตัดออกเมื่อยืนยันว่า MAX30102 เสถียรในการใช้งานจริง
5. runbook ตั้งเครื่อง (`docs/carekeeper-pi5-setup-runbook.md`) ยังไม่มีขั้นตอนเปิด 1-Wire สำหรับ DS18B20 และการต่อสาย MAX30102 — ควรเพิ่มก่อนตั้งเครื่องใหม่ครั้งต่อไป
6. API key ต้องตั้งค่าให้ถูกต้องก่อนใช้งานจริง — `.env` บน Pi ต้องไม่ใช่ค่าทดสอบ (test) และห้าม commit `.env` เข้า git
7. ควรทดสอบบน Raspberry Pi จริงหลังแก้ UI ทุกครั้ง — หน้าจอขนาดเฉพาะ และ touchscreen/on-screen keyboard แสดงผลต่างจาก Windows
8. Log ยังพิมพ์ไปที่ console เท่านั้น — ควรต่อเข้ากับ `journald` (`conf/carekeeper-journal.conf` จำกัดขนาดไว้แล้ว) หรือเขียนไฟล์ log แยก
9. mock mode ใช้ queue path เดียวกับ real mode — ถ้ามี pending payload จริงค้างอยู่แล้วเปิด `main_demo.py` อาจทำให้ mock worker drain/delete ข้อมูลจริงได้ ควรแยก queue path
10. `pyqtgraph` อยู่ใน `requirement.txt` แต่ยังไม่ได้ถูกเรียกใช้จริงในโค้ด — ถ้ายังไม่ทำกราฟ ควรถอดออกเพื่อลดขนาดการติดตั้ง

